UVA11475 Extend to Palindrome

题目描述

输入多个字符串。

对于每个字符串 S ，求出一个字符串 S $*$
需要满足：

S 为 S $*$的前缀；
S $*$是一个回文字符串；
S $*$应尽可能小；
对于每个 S ，输出 S $*$。

你的思路是**非常正确且高效的**。使用 Manacher 算法确实是解决此类问题的最优方案。

不过，为了避免在“枚举切分点”时导致复杂度过高（比如如果直接暴力枚举切分点并遍历去判断回文，复杂度可能会退化到 $O(n^2)$），我们可以把你的思路稍微优化一下，使其达到严格的 $O(n)$ **线性时间复杂度**。

你的思路核心

Manacher 算法能够预处理出每个中心点对应的最长回文半径。你提到的“利用包含关系来判断回文”，本质上就是：如果以某个中心点为圆心的最长回文半径为 R，那么对于任意 $r \leq R$，该区间也是回文串。

优化后的线性解法步骤

题目要求寻找最长的双回文子串 $T = X + Y$。这其实等价于寻找一个切分点 i，使得：

1. X 是以 i 结尾的最长回文串。
2. Y 是以 $i+1$ 开头的最长回文串。
3. 最终结果为 $\max(|X| + |Y|)$。

你可以按照以下步骤来实现：

1. Manacher 预处理

利用 Manacher 算法求出原字符串中每个位置对应的回文半径数组 `p[i]` （通常为了处理偶数长度回文，会在字符间插入 `#` ）。

2. 预处理两个辅助数组 (这是关键)

你需要两个数组来避免重复的枚举和判断：

- **`left_max[i]`**： 表示以位置 `i` **结尾**的最长回文串的长度。
- **`right_max[i]`**： 表示以位置 `i` **开始**的最长回文串的长度。

计算方式：

- 在 Manacher 遍历过程中，对于每一个中心点及其回文半径，它覆盖了一个区间 `[start, end]` 。
- 对于这个区间， `left_max[end]` 至少可以是 `length` （如果之前已经有更长的，取最大值）。
- 同理更新 `right_max[start]` 。
- **注意**：这里需要有一个向外扩散或更新的过程，因为回文串的中心可能在 i 的左边，但它的右边界在 i。简单的做法是：遍历完 Manacher 后，进行两次线性扫描（一次从左到右维护 `left_max` ，一次从右到左维护 `right_max` ），确保所有包含在内的回文子串都被更新到。

3. 结果合并

最后，你只需要遍历一遍所有的分割点 i (从 0 到 $n-2$)：

$$\text{ans} = \max_{0 \le i < n-1} (left_max[i] + right_max[i+1])$$

为什么这样做比你的“枚举”更好？

- **你的原始设想**：对每个切分点 i，去判断 $S[0..i]$ 是否是回文，去判断 $S[i+1..n-1]$ 是否是回文。这通常意味着你要分别判断左半部分的最长回文后缀和右半部分的最长回文前缀。
- **优化后的设想**：通过预处理 `left_max` 和 `right_max` ，我们在 $O(1)$ 的时间内就能知道以 i 结尾的最长回文长度是多少，不需要每次都重新计算或判断。

小贴士

- **Manacher 的处理**：记得处理字符串的预处理（加 `#` ），这样奇偶回文可以统一处理。
- **下标映射**：计算回文长度时，注意 `#` 插入后下标和原字符串下标的映射关系，这是最容易出错的地方。

总结来说，你的直觉非常敏锐，这就是解决这道题的标准正解路径。只要把“枚举判断”改为“预处理数组”，代码就能在 $O(n)$ 时间内完美跑通。